

International Institute of Information Technology, Bangalore
CSE 511 Algorithms: Practice Problem

Description: You are given two 0-indexed arrays `nums1` and `nums2`, both of length n , and both are permutations of the integers $\{0, 1, 2, \dots, n-1\}$.

A triplet (x, y, z) is called a *good triplet* if all of the following hold:

- (i) x, y, z are pairwise distinct.
- (ii) The positions of x, y, z in `nums1` appear in increasing order, i.e.

$$\text{pos1}[x] < \text{pos1}[y] < \text{pos1}[z],$$

where $\text{pos1}[v]$ denotes the index of value v in `nums1`.

- (iii) The positions of x, y, z in `nums2` appear in increasing order, i.e.

$$\text{pos2}[x] < \text{pos2}[y] < \text{pos2}[z],$$

where $\text{pos2}[v]$ denotes the index of value v in `nums2`.

Return the number of good triplets.

Input:

- The first line contains a single integer n , the length of both permutations.
- The second line contains n integers: `nums1`.
- The third line contains n integers: `nums2`.

Output: Print a single integer — the number of good triplets.

Example 1:

Input:

5

2 0 1 3 4

0 1 2 3 4

Output:

4

Explanation: The good triplets (x, y, z) are:

$$(2, 3, 4), \quad (1, 3, 4), \quad (0, 3, 4), \quad (0, 1, 4).$$

The tables below show positions in `nums1` and `nums2`:

value	0	1	2	3	4
pos1	1	2	0	3	4
pos2	0	1	2	3	4

For example, (2, 3, 4): $\text{pos1}[2] = 0 < \text{pos1}[3] = 3 < \text{pos1}[4] = 4$, and $\text{pos2}[2] = 2 < \text{pos2}[3] = 3 < \text{pos2}[4] = 4$. Hence it is good.

Example 2:

Input:

```
4
4 0 1 3 2
4 1 0 2 3
```

Output:

```
0
```

Explanation: No triplet satisfies both orderings, so the answer is 0.

Constraints:

- $3 \leq n \leq 10^5$
- `nums1` and `nums2` are permutations of $\{0, 1, \dots, n-1\}$.

Required approach: Relabel each value by its position in `nums1`. Then `nums2` becomes a permutation array P where $P[i] = \text{pos1}[\text{nums2}[i]]$. A good triplet corresponds to indices

$$i < j < k \quad \text{with} \quad P[i] < P[j] < P[k].$$

For each middle index j :

$$\text{leftLess}[j] = \#\{i < j : P[i] < P[j]\}, \quad \text{rightGreater}[j] = \#\{k > j : P[k] > P[j]\}.$$

The answer is

$$\sum_{j=0}^{n-1} \text{leftLess}[j] \times \text{rightGreater}[j].$$

Each of `leftLess` and `rightGreater` is computed with a **Fenwick tree (BIT)** over values in $O(n \log n)$.

Complexity:

- Time: $O(n \log n)$.
- Space: $O(n)$.

End of Problem